

Module-4

Part-2

Autonomous Map Building (SLAM)

Autonomous Map Building (SLAM)

- **Localization** means The robot already has a map and must find its position in that map.
- But what if:
 - The robot **does not have a map**
 - The environment is **unknown**
- Then the robot must:
 - **Build the map**
 - **Find its own position**at the **same time**.
- This is called **Simultaneous Localization and Mapping (SLAM)**.

Why SLAM is Difficult

- There is a circular dependency.
- To **build a map**, the robot must know its **position**.
- But to **know its position**, the robot must already have a **map**.
- So both problems depend on each other.
- This is why SLAM was historically considered **very difficult**.

The SLAM Problem

- The SLAM problem is formally defined as:
 - The robot must estimate:
 - **its trajectory**
 - **positions of landmarks****from sensor observations.**

Variables Used in SLAM

Symbol	Name	Meaning	Example / Explanation
x_t	Robot pose	Position and orientation of the robot at time t	$x_t = (x, y, \theta)$
$x_{0:t}$	Robot trajectory	Sequence of robot poses from time 0 to t	Entire path of the robot
m	Map	Representation of the environment	Set of landmarks
m_i	Landmark position	Position of the i-th landmark in the map	Wall corner, pole, feature
u_t	Control input	Motion command applied to the robot at time t	Move forward, rotate
$u_{1:t}$	Control history	Sequence of control inputs from time 1 to t	All past movements
z_t	Observation	Sensor measurement at time t	Laser distance, camera feature
$z_{1:t}$	Observation history	All sensor measurements until time t	Entire sensor data
$p(x_t, m \mid z_{1:t}, u_{1:t})$	SLAM posterior	Probability of robot pose and map given observations and controls	Main SLAM estimation

Goal of SLAM

Compute:

$$p(x_t, m \mid z_{1:t}, u_{1:t})$$

The probability of the robot pose x_t and the map m , given the observations $z_{1:t}$ and control inputs $u_{1:t}$

The robot is trying to estimate:

- Where am I?
- What does the environment look like?

Using:

- What I sensed
- How I moved

So SLAM tries to compute:

Robot position + Map

based on

Sensors + Movements

The robot cannot know exact values(uncertainty). Instead it estimates **probability distributions**.

Mathematical definition of SLAM

$$p(x_t, m \mid z_{1:t}, u_{1:t})$$

How the robot updates this probability

- SLAM updates this probability **every time step**.
- Two main steps(definition):
 1. Prediction Step (Motion Update)
 2. Correction Step (Measurement Update)

Prediction Step (Motion Update)

$$x_t = f(x_{t-1}, u_t)$$

- The robot predicts its **new position** based on:
 - **previous position**
 - **control input (movement)**
- Example control input:
 - wheel rotations
 - move forward
 - Turn
- Prediction model answers **If the robot moves with control u_t , where will it probably be?**
- Output of prediction model is **Predicted robot pose**

Correction Step (Measurement Update)

$$p(z_t | x_t, m)$$

This describes what the robot should observe.

Inputs of Measurement Model:

x_t

Robot pose

m

Map (landmark positions)

Output of Measurement Model

z_t

Predicted observation

Example observation:

- distance to wall
- angle to landmark
- camera feature location

Suppose a robot has a **laser sensor**.

There is a landmark at:

$$m_1 = (5, 2)$$

Robot is at:

$$x_t = (2, 2)$$

Distance between them:

3 meters

So the **expected measurement** is:

3 meters

This expected value is produced by the **measurement model**.

Measurement model answers **If the robot is at pose x_t and the landmark is at m , what should the sensor measure? Measurement model computes this from an algorithm not from sensors.**

- **Predicted measurement/observation** means what the robot expects the sensor should observe based on the current map and robot pose.

- Suppose the robot believes:

Robot position = (2,3)

Landmark position = (6,7)

Using the measurement model, the robot predicts what the sensor should see. It is different from actual sensor reading.

Prediction uses **motion model**

$$x_t = f(x_{t-1}, u_t)$$

Measurement prediction uses **measurement model**

$$\hat{z}_t = h(x_t, m)$$

Actual measurement comes from **sensors**

$$z_t$$

Full flow:

- Start
- Initialize pose and map
- Repeat:
 - Move robot (control input)
 - Prediction step
 - Predict new pose using motion model
 - Sense environment
 - Get sensor measurements
 - Measurement prediction
 - Predict expected observation
 - Compare predicted and actual measurement
- update step
 - Update robot pose
 - Update map
- Until mission finished

- SLAM operates in a recursive cycle consisting of prediction using the motion model, sensing the environment, predicting expected observations using the measurement model, comparing them with actual measurements, and updating both the robot pose and the map.

Extended Kalman Filter (EKF)

- A **Kalman Filter** is an algorithm used to **estimate the true state of a system** by combining:
 - **predictions from a model**
 - **noisy sensor measurements**
- It produces the **best possible estimate** even when data contains uncertainty.

Example :

- A robot wants to know its **position**, but:
 - sensors have **noise**
 - motion has **errors**
- Kalman filter gives the **best estimate of the robot position**.

Why Kalman Filter is Needed

- In real-world systems:
 - Sensors are **not perfect**
 - Measurements contain **noise**
 - Motion models are **not exact**
- Example: A robot tries to estimate its position using:
 - wheel encoders
 - GPS
 - camera

But all these sensors have **errors**.

- The Kalman Filter helps to **reduce these errors and estimate the most accurate state.**

Why Do We Need Extended Kalman Filter?

- The **standard Kalman Filter** works only for **linear systems**.
- But in robotics:
 - Motion equations are **nonlinear**
 - Measurement equations are **nonlinear**

Examples:

- robot rotation
- camera projection
- SLAM measurements
- So we use:
 - **Extended Kalman Filter (EKF)**
 - EKF can handle **nonlinear models**.

- The main use of Kalman Filter is **To estimate unknown variables of a dynamic system in the presence of noise and uncertainty.**
- Extended Kalman Filter is a recursive estimation algorithm used to estimate the state of nonlinear dynamic systems by combining prediction and noisy measurements.
- EKF handles **nonlinear equations** by **linearizing them using Jacobians.**

Jacobians

- A **Jacobian** is a **matrix of partial derivatives** that describes **how a function changes when its input variables change**.

For a function

$$f(x, y) = \begin{bmatrix} f_1(x, y) \\ f_2(x, y) \end{bmatrix}$$

The Jacobian matrix is

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{bmatrix}$$

Each element represents a **partial derivative**.

x_t	Estimated state vector at time t
$\hat{x}_t$	Predicted state at time t
P_t	Covariance matrix representing uncertainty of the state
u_t	Control input (robot motion command)
z_t	Sensor measurement
$f(\cdot)$	Nonlinear motion model
$h(\cdot)$	Nonlinear measurement model
F_t	Jacobian of motion model
H_t	Jacobian of measurement model
Q_t	Process noise covariance
R_t	Measurement noise covariance
K_t	Kalman Gain
I	Identity matrix

Extended Kalman Filter algorithm

EKF Algorithm

1. Initialize robot state and uncertainty.
2. Predict the next state using the motion model.
3. Increase uncertainty due to motion noise.
4. Receive sensor measurement.
5. Compare predicted measurement with actual measurement.
6. Compute Kalman Gain.
7. Correct the predicted state.
8. Update the uncertainty.
9. Repeat the process for the next time step.

Example: Uncertainty

Robot position = 6 m

we say:

Robot position $\approx 6 \text{ m} \pm 0.5 \text{ m}$

Means

The robot is **somewhere near 6 meters**, but not exactly known.

That $\pm$ value represents **uncertainty**.

- EKF represents uncertainty using a **covariance matrix P** .
- P_t -Uncertainty of the state estimate at time t (**Instead of a single point, we represent position as a **region**.**)

Step 1 — Initialization

Initialize the system state and uncertainty.

$$x_0 = \text{initial state estimate}$$
$$P_0 = \text{initial covariance matrix}$$

Step 2 — Prediction of State

Predict the state at the next time step using the motion model.

$$\hat{x}_t = f(x_{t-1}, u_t)$$

Where

- $f(\cdot)$ is the **nonlinear motion model**
- u_t is the **control input**

Step 3 — Prediction of Covariance

Update the uncertainty of the predicted state.

$$P_t = F_t P_{t-1} F_t^T + Q_t$$

Where

- F_t = Jacobian of motion model
- Q_t = process noise covariance

Symbol	Meaning
P_t	Predicted covariance (new uncertainty after motion)
P_{t-1}	Previous covariance (previous uncertainty)
F_t	Jacobian of the motion model
F_t^T	Transpose of the Jacobian
Q_t	Process noise covariance (uncertainty from motion noise)

Step 4 — Measurement Acquisition

Receive measurement from sensors.

$$z_t$$

Example sensors:

- camera
 - LiDAR
 - GPS
-

Step 5 — Compute Kalman Gain

Kalman Gain determines how much the prediction should be corrected.

$$K_t = P_t H_t^T (H_t P_t H_t^T + R_t)^{-1}$$

Where

- H_t = Jacobian of measurement model
 - R_t = measurement noise covariance
-

Step 6 — State Update

Correct the predicted state using the measurement.

$$x_t = \hat{x}_t + K_t(z_t - h(\hat{x}_t))$$

Where

- $h(\hat{x}_t)$ = predicted measurement

Step 7 — Covariance Update

Update the uncertainty after correction.

$$P_t = (I - K_t H_t) P_t$$

Visual SLAM with a single camera(Monocular V-SLAM)

- **Visual SLAM (V-SLAM)** is a method where a robot uses **only a camera** to:
 - **Estimate its own position (localization)**
 - **Build a map of the environment**
- It was introduced by **Andrew Davison in 2003**.
- Unlike other SLAM methods, **no odometry, GPS, or laser sensors are used.**
- **Only one camera is used.**
- So the robot learns everything **only from images.**

Monocular SLAM is Difficult

- In **laser SLAM**, sensors measure:
 - angle
 - distance
- But The camera only knows **the direction of the feature**, not the **distance**.
- Therefore **Estimating feature depth is harder**.

- In **monocular V-SLAM**, we use **only one camera**. And a **single camera cannot measure distance**.

Example :

- The camera sees a corner in the image.
- But that corner could be:
 - 2 m away
 - 5 m away
 - 10 m away
- The camera **cannot know the distance** .So at the beginning of SLAM, we **cannot compute the 3D position of features**.This is the **initialization problem/depth ambiguity problem**.

Davison's Idea to Solve This Problem

- Davison solved this by putting a **known reference object** in front of the camera.
- This is simply a **flat board with known corner positions**.



- The **shape and size of the pattern are known**. The system knows the **exact distances between corners**.
- Note: A **pattern** means a **known reference object (physical object not virtual) placed in front of the camera** whose **shape and corner positions are already known**.
- It is usually a **flat printed design** used so the computer can easily detect specific points.

09-03-2026 **pattern = reference object with known feature points.**

- Now the camera captures an image which has the reference object in it. (3D coordinates of the object are already known).
- And from the image, we get 2D coordinates of reference object.
- So now we have **two sets of information**:
 - 3D world coordinates of points
 - +
 - 2D image coordinates of the same points
- **Solve the Pose Estimation Problem:** Where must the camera be located so that these 3D points appear at these 2D positions in the image?
- By solving this **geometric relationship**, the algorithm computes:
 - camera position (x,y,z)
 - camera orientation (direction)
- This is called: **Camera Pose Estimation.**

- Because the system knows reference object's/pattern geometry, it can compute:

camera position

camera orientation

when the camera sees the pattern.

This helps initialize the SLAM system.

- i.e From four corners of known position, the 6DoF camera pose with respect to these points can be determined uniquely.

Note :A **pattern** is a known reference object with identifiable feature points whose positions are already known.

```
Known pattern geometry
+
Detected corner positions in image
↓
Solve projection equations
↓
Compute camera position and orientation
```

As the camera starts moving away from the pattern, new features must be triangulated and added to the map.

At this point, the EKF V-SLAM process starts.

Triangulation calculates the position of an object by using **multiple observations from different locations**.

triangulation

- Camera position 1 → sees feature
Camera position 2 → sees same feature
- Now the system has **two viewing directions**.

From each camera position, we can draw a **line pointing toward the feature**.



Each camera gives a **direction line**.

4. Where the Lines Meet

The intersection of the two lines gives the **actual location of the feature**.



These three points form a **triangle**.

That is why it is called:

Triangulation

EKF used in V-SLAM

- Davison used an **Extended Kalman Filter (EKF)**.
- EKF estimates:
 - camera position
 - map features

while handling **uncertainty**.

State Vector in V-SLAM

The equation

$$y_t = [x_t, m_0, m_1, \dots, m_{n-1}]^T$$

represents the **complete EKF state vector**.

It contains:

1. camera state
2. map features

The camera state is defined separately as:

$$\mathbf{x}_t = [r_t, q_t, v_t, \omega_t]^T$$

This includes four components.

Symbol	Meaning
r_t	camera position
q_t	camera orientation
v_t	camera linear velocity
ω_t	camera angular velocity

EKF

EKF always works in **two main steps**:

- **Prediction step**
- **Measurement update step((Correction)**

Input and output of prediction step

Input	Explanation
Previous state x_{t-1}	Contains the previous estimate of the system including camera position, orientation, velocity, and map feature positions.
Previous covariance P_{t-1}	Represents the uncertainty associated with the previous state estimate.
Motion model $f(\cdot)$	Mathematical model used to predict camera motion. In V-SLAM a constant velocity model is commonly used.
Process noise Q_t	Represents uncertainty in motion prediction caused by acceleration, camera shake, or other disturbances.
Output	Explanation
Predicted state $\hat{x}_t$	Estimated camera pose and velocity at the next time step before using new observations.
Predicted covariance P_t	Updated uncertainty of the predicted state after motion prediction.

Input and output of correction step

Input	Explanation
Predicted state $\hat{x}_t$	The predicted camera pose and map features obtained from the prediction step.
Predicted covariance P_t	Represents the uncertainty of the predicted state.
Camera measurements z_t	Observed feature positions detected in the current camera image.
Measurement model $h(\cdot)$	Predicts how map features should appear in the image given the camera pose.
Measurement noise R_t	Represents uncertainty in sensor observations such as feature detection errors or image noise.
Output	Explanation
Updated state x_t	Corrected estimate of camera pose and feature positions after incorporating the observations.
Updated covariance P_t	Reduced uncertainty after the correction step.

Prediction

No odometry in V-SLAM

Use **constant velocity model**.

The system predicts where the camera **should move** based on its **previous velocity**.

Velocity : How fast and in which direction the camera is moving.

The motion prediction function f is defined as:

$$\hat{x}_t = f(x_{t-1}, u_t) = \begin{bmatrix} r + (v + V)\Delta t \\ q \times q((\omega + \Omega)\Delta t) \\ v + V \\ \omega + \Omega \end{bmatrix}, \quad (5.113)$$

Term	Meaning
r	position
v	velocity
ω	angular velocity
Δt	time step
V	translation noise
Ω	rotation noise

Motion is not perfect.
So Davison adds **noise terms**.

Noise Model

$$V = a\Delta t$$

$$\Omega = \alpha\Delta t$$

Where

Symbol

Meaning

a

translational acceleration

α

angular acceleration

They are modeled as **zero mean Gaussian noise**.

EKF Prediction Update

- During prediction:
 - Camera pose is predicted
 - Feature positions stay the same(camera pose → changes, features → unchanged).Because features are **fixed in the world**.

Measurement Update

Now the camera captures a new frame.

`</>` Plain text

```
Frame t-1 → prediction
Frame t   → observation
```

The system detects features in the new image.

- In the measurement update, the camera pose is corrected based on the reobservation of feature.
- By observing the features again and again, the system adjusts the camera position and orientation.
- In the measurement update step, the predicted state is corrected using sensor observations(camera).
- The camera pose and feature positions are updated based on the difference between predicted and observed feature measurements.

Overview of SLAM algorithm

Step 1 — Initialization

Place a **planar pattern** or **known reference object** in front of the camera.

Detect at least **four known feature points** on the pattern.

Compute the **initial camera pose (position and orientation)** using the known 3D coordinates of the pattern corners and their image positions.

Step 2 — Capture Image

The camera captures the **current image frame** of the environment.

Step 3 — Feature Detection

Detect **distinctive visual features** in the image such as:

- corners
- edges
- textured points

These features will act as **landmarks for mapping**.

Step 4 — Prediction (Motion Model)

Predict the **next camera state** using the **constant velocity model**.

Estimate the **camera pose (position and orientation)** based on previous motion.

Also update the **uncertainty of the prediction**.

Step 5 — Feature Observation

Observe the **features in the new image frame**.

Match them with **previously stored map features**.

Step 6 — Measurement Update (EKF Correction)

Use the observed feature positions to:

- correct the **predicted camera pose**
- refine the **feature positions in the map**

This step reduces the **estimation error**.

Step 7 — Triangulation of New Features

If new features appear:

1. observe them from **multiple camera positions**
 2. compute their **3D coordinates using triangulation**
 3. add them to the **map**
-

Step 8 — Map Update

Update the **map of the environment** with:

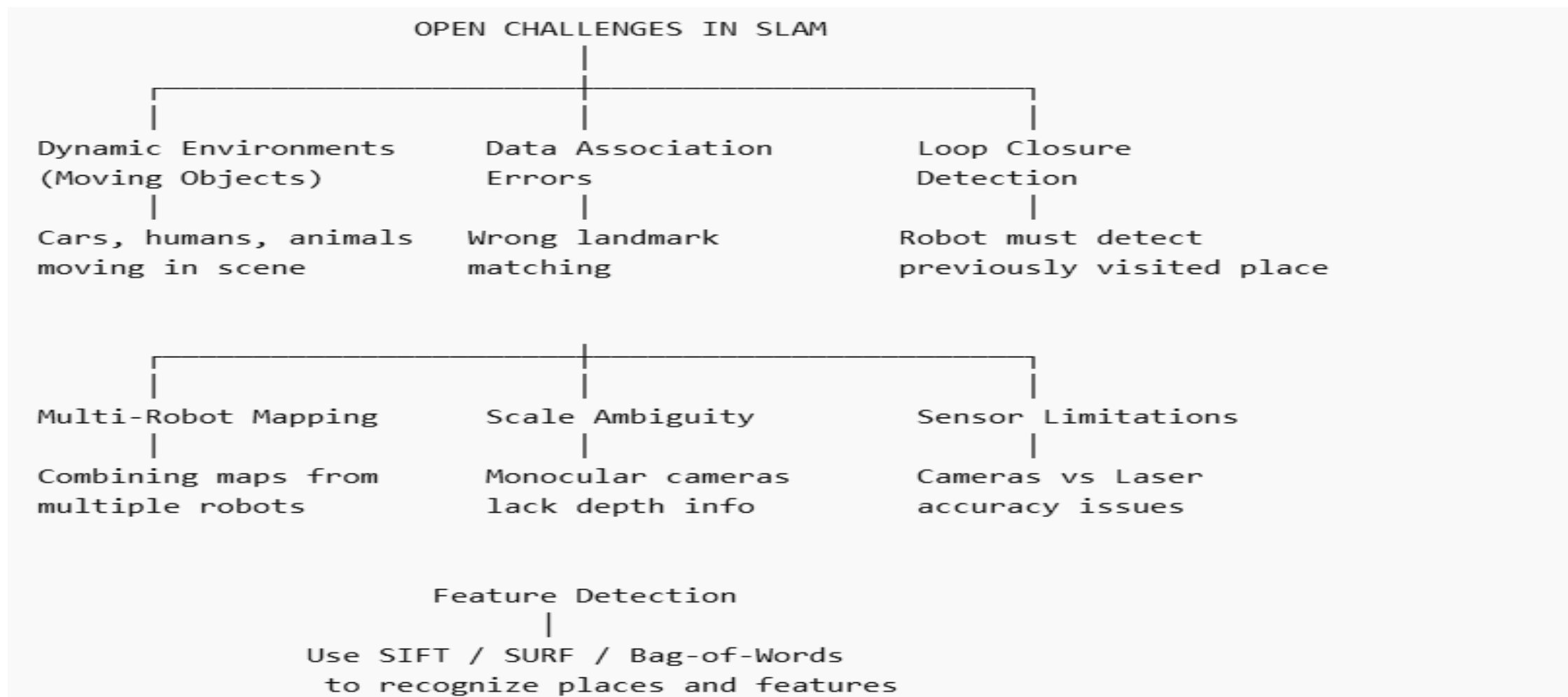
- corrected feature positions
 - newly triangulated features
-

Step 9 — Repeat

Repeat Steps 2–8 for every new image frame to continuously estimate:

- camera trajectory
- map of the environment

Open challenges in SLAM



Dynamic Environments (Moving Objects)

- In many SLAM algorithms, we assume: The map is static (objects do not move).
- But in real environments there are moving objects such as:
 - cars
 - humans
 - animals
 - bicycles
- **Problem** :Moving objects can confuse the SLAM algorithm.

Example: If a person walks across the camera view, the system might think that the **map itself is changing**.

- **Possible Solution**
 - Treat moving objects as **outliers** (ignore them).
- Better approaches try to:
 - detect moving objects
 - track their motion
 - remove them from the map

Multi-Robot Mapping

- Another research challenge is: Multiple robots building the same map together.
- Example:
 - several robots exploring a building
 - each robot collects its own data
- **Problem** :We must combine:
 - maps from different robots
 - sensor data from different viewpoints
- **Challenges**
 - map merging
 - robot coordination
 - communication between robots
- **Benefit** :Multiple robots can **explore large environments faster**.

Incorrect Data Association

- SLAM must match: current observations $\leftrightarrow$ previously seen landmarks

This is called **data association**.

Problem

- Sometimes the system matches the **wrong features**.

Example:

- The robot sees two similar doors and thinks they are the same door.

This causes:

- incorrect map
 - wrong robot position
- This problem becomes serious during **loop closure**.

Loop Closure Problem

- Loop closure occurs when: Robot returns to a previously visited place.
Example: Robot moves in a large circle and comes back to the start.

Problem

- Because of accumulated errors, the robot might think it is **somewhere else**.

So the system must detect:

- "Yes, I have been here before."

Why It Is Difficult

- environments may look similar
- sensor noise
- large travel distances

Feature Detection in Large Environments

- To solve loop closure problems, SLAM uses **distinctive features**.

Examples:

- SIFT (Scale Invariant Feature Transform)
- SURF (Speeded Up Robust Features)
- These features can be detected even when:
 - camera viewpoint changes
 - object scale changes
 - lighting changes
- Another technique used is:

Bag of Features Approach

- Images are represented as **collections of visual features**, which helps recognize previously visited places.

Sensor Limitations

- Different sensors have different advantages and disadvantages.

Laser Sensors

- Advantages:
 - accurate distance measurements
- Disadvantages:
 - expensive
 - heavy
 - limited visual information

Cameras

- Advantages:
 - cheap
 - lightweight
 - provide rich visual information
- Disadvantages:
 - monocular cameras cannot measure depth directly

Stereo Camera Limitation

- Stereo cameras can estimate **depth directly**.
- But they have a drawback:
 - Depth accuracy decreases as distance increases.
 - So far-away objects are measured less accurately.